

İZMİR UNIVERSITY OF ECONOMICS

SE 410 - SOFTWARE FRAMEWORK APPLICATIONS PROJECT REPORT



Göksun Gürel - 20220601039

Eda Naz Soytürk - 20230601063

Enes Deniz - 20220601017

Barış Dursun - 20220601022

Content

1) Introduction	3
1.1) Project Overview	3
1.2) Motivation and Problem Statement	3
1.3) Project Objectives.....	4
2) Requirements	4
2.1) Functional Requirements	4
2.2) Non-Functional Requirements	5
3) Database Schema	6
3.1) SQL.....	7
3.2) Connection Configuration.....	8
3.3) Entity Tables and Relationships	9
3.4) DbSet Declarations	10
4) LINQ Queries.....	10
4.1) LINQ Filtering & Ordering (Today's Schedule)	10
4.2) LINQ Search Functionality (Client Registry)	11
4.3) LINQ Aggregating (Financial & Operational Reporting)	11
4.4) LINQ Relational Mapping (One-to-Many Isolation)	12
5) Delegates and events	12
5.1) Architectural Purpose and Functionality	13
6) MVC Structure	14
7) Sources	15
7.1) Source Code Repository (GitHub)	15
7.2) Installation and Execution Guide	15
7.3) Video Demonstrations	16
8) User Interface Design.....	16
9) Tests.....	16
9.1) Test Infrastructure.....	19
9.2) Test Cases.....	19
10) Conclusion	21

1)Introduction

1.1) Project Overview

In recent years, the pet care and veterinary industry has experienced significant growth, leading to a substantial increase in the operational workload of clinics and grooming centers. Managing daily clinical schedules, maintaining detailed health records, tracking grooming sessions, and handling financial billing simultaneously has become a challenging task for clinic staff. Traditional pen-and-paper methods or fragmented software solutions often fail to handle the rapid data flow, leading to communication gaps, double-booked appointments, and a lack of transparency for pet owners.

The Pet Grooming and Veterinary Appointment System (PetClinicApp) has been designed and implemented to address these specific operational inefficiencies. The system introduces a modern, dual-platform software architecture tailored to bridge the gap between service providers and customers. By utilizing a Desktop application for internal clinic administration and a Web application for client interaction, the project delivers a unified ecosystem that streamlines the entire workflow of a contemporary veterinary and grooming facility.

1.2) Motivation and Problem Statement

The primary motivation behind this project stems from the administrative challenges observed in pet clinics. Pet care requires strict tracking; a missed vaccination, an overlooked grooming detail, or a delayed medical note can directly affect an animal's well-being. From an administrative perspective, clinic receptionists deal with thousands of client records, making manual searches highly inefficient.

Furthermore, today's pet owners demand instant access and transparency regarding their pets' care journeys. Allowing customers to seamlessly request appointments online and instantly notifying the clinic staff eliminates manual overhead and prevents scheduling conflicts. Therefore, developing a synchronized framework that handles robust data management, real-time platform communication, and data aggregation is an essential engineering necessity for professional clinic operations.

1.3) Project Objectives

The main objective of this project is to develop a reliable, cross-platform scheduling and management system using modern software engineering principles and C# technologies. The specific goals established for PetClinicApp include:

- **Centralized Desktop Administration:** Providing clinic staff with a robust desktop interface to perform full CRUD operations on client and pet records, ensuring a seamless data management experience.
- **Dynamic Web-Based Scheduling:** Creating an intuitive web platform where customers can securely log in, view their registered pets, and send targeted appointment requests for specific services (Veterinary or Grooming).
- **Real-Time Platform Synchronization:** Utilizing advanced C# Delegate and Event mechanisms to establish an instant notification pipeline, ensuring that web-initiated actions are immediately visible on the desktop panel.
- **Advanced Data Querying and Reporting:** Implementing complex LINQ queries to allow rapid searching across thousands of database entries, alongside LINQ Aggregation to dynamically generate daily financial reports and operational dashboard metrics.
- **Architectural Separation and Testing:** Adhering to the Model-View-Controller (MVC) design pattern for the web component to maintain clean code separation, backed by thorough Unit Testing to guarantee the reliability of the system's core business logic.

2) Requirements

2.1 Functional Requirements (FR)

Functional requirements define the specific behaviors, attributes, and core functionalities that the PetClinicApp system must execute.

- **FR-1: Client and Pet Management (Desktop - CRUD):** Staff must be able to create, update, and delete records for new clients and their pets within the clinic database.
- **FR-2: Pet Profiling (Desktop):** Staff shall be able to define multiple pets (name, species, age, gender) for a single client and establish a one-to-many relationship between them.
- **FR-3: Customer Appointment Panel (Web):** Customers must be able to log in via the web interface to schedule "Veterinary" or "Grooming" appointments by selecting their registered pet and an available time slot.

- FR-4: Appointment Status Management (Desktop): Staff shall have the authority to update appointment statuses as "Pending", "Completed", or "Cancelled" for both web-initiated and manual entries.
- FR-5: Dynamic Appointment Filtering (LINQ - Desktop): Staff must be able to perform fast filtering within the appointment list (e.g., "Today's Appointments" or "Grooming Only") using LINQ queries.
- FR-6: Personalized Appointment Tracking (LINQ - Web): Customers shall be able to view their own pets' past and upcoming appointments via the web panel, filtered using LINQ.
- FR-7: Digital Pet Record & Clinical Notes (Desktop): Upon completion of an appointment, staff must be able to add persistent medical or grooming notes (e.g., "Rabies vaccine administered") to the pet's profile.
- FR-8: Service Billing and Payment Entry (Desktop): Staff shall be able to enter service fees and record payment data for appointments marked as "Completed".
- FR-9: Real-Time Notification Triggers (Delegates): The system must use C# Delegate/Event structures to trigger visual alerts on both platforms whenever a new appointment is booked or cancelled.
- FR-10: Operational Dashboard (Desktop): Upon startup, the application shall display a summary of daily total appointments, pending tasks, and estimated daily revenue.

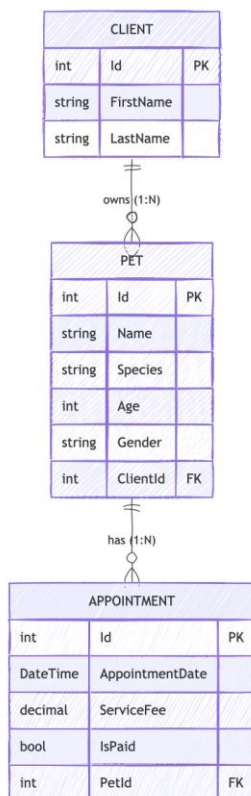
2.2 Non-Functional Requirements (NFR)

Non-functional requirements specify the quality attributes, performance benchmarks, security constraints, and architectural limitations of the system.

- NFR-1: Data Synchronization (Consistency): An appointment booked via the Web interface must be visible on the Staff Desktop panel within a maximum of 2 seconds.
- NFR-2: Access Control (Authorization): While customers can only access their own pet data, financial records and the full clinic calendar must be restricted to authorized staff (Role-Based Access Control).

- NFR-3: Query Performance: LINQ search and listing operations must return results in under 2 seconds, even if the total database records exceed 5,000.
- NFR-4: User Experience (UI/UX): The system must support keyboard shortcuts for rapid data entry by staff, while the customer interface must be fully Responsive for mobile devices..
- NFR-5: Robust Error Handling (Exception Handling): In the event of database connection failures or invalid data entry, the system must not crash and should display error messages instead of technical codes.

3) Database Schema



The database layer of PetClinicApp is built on SQLite using Entity Framework Core as the Object-Relational Mapper. The central component of this layer is the AppDbContext class, which inherits from DbContext and manages three core entity sets: Clients, Pets, and Appointments. Upon the application's first execution, the Database.EnsureCreated() call

automatically provisions the database file, eliminating the need for manual migration scripts during development and test environments.

3.1) SQL

To supplement the visual architecture and fully satisfy the database definition blueprints, the comprehensive SQL Data Definition Language scripts executed during the database initialization pipeline are provided below:

```
CREATE TABLE "Clients" (  
    "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
    "FirstName" TEXT NOT NULL,  
    "LastName" TEXT NOT NULL,  
    "PhoneNumber" TEXT NOT NULL,  
    "Email" TEXT NOT NULL  
);  
  
CREATE TABLE "Pets" (  
    "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
    "Name" TEXT NOT NULL,  
    "Species" TEXT NOT NULL,  
    "Age" INTEGER NOT NULL,  
    "Gender" TEXT NOT NULL,  
    "ClinicalNotes" TEXT NOT NULL,  
    "ClientId" INTEGER NOT NULL,  
    CONSTRAINT "FK_Pets_Clients_ClientId" FOREIGN KEY ("ClientId")  
        REFERENCES "Clients" ("Id") ON DELETE CASCADE  
);  
  
CREATE TABLE "Appointments" (  
    "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
    "AppointmentDate" TEXT NOT NULL,  
    "Type" INTEGER NOT NULL,  
    "Status" INTEGER NOT NULL,  
    "ServiceFee" TEXT NOT NULL,
```

```

        "IsPaid" INTEGER NOT NULL,
        "PetId" INTEGER NOT NULL,
        CONSTRAINT "FK_Appointments_Pets_PetId" FOREIGN KEY ("PetId")
            REFERENCES "Pets" ("Id") ON DELETE CASCADE
    );

```

3.2) Connection Configuration

The `OnConfiguring` method dynamically resolves the database file path by targeting the `Environment.SpecialFolder.LocalApplicationData` directory rather than a relative path. This design decision was made to resolve a platform-specific issue where relative path usage caused an *"unable to open database file"* error on macOS environments. By resolving a writable system folder at runtime and invoking `Directory.CreateDirectory()` to guarantee its existence, the configuration remains fully portable across Windows desktop, macOS, and the ASP.NET web host without any environment-specific adjustments.

```

0 references
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    var folder = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData);
    Directory.CreateDirectory(folder);
    var dbPath = Path.Combine(folder, "petclinic.db");
    optionsBuilder.UseSqlite($"Data Source={dbPath}");
}

```

You, last month • AppDbContext.cs ...

3.3) Entity Tables and Relationships

The schema consists of three interrelated tables connected through foreign key constraints, forming a strict relational hierarchy.

The **CLIENT** table serves as the top-level entity, storing core owner information through three fields: `Id` (PK, int), `FirstName` (string), and `LastName` (string).

The **PET** table sits one level below the client, storing the animal profiles registered under a given owner. Its fields include Id (PK, int), Name, Species, Age, Gender, and ClientId (FK → CLIENT). Since a single client may own multiple animals, the CLIENT–PET relationship is defined as **one-to-many (1:N)**.

The **APPOINTMENT** table occupies the lowest tier of the hierarchy and captures all scheduling and billing data associated with a specific pet. Its fields include Id (PK, int), AppointmentDate (DateTime), ServiceFee (decimal), IsPaid (bool), and PetId (FK → PET). Given that a single pet may have multiple appointments over its lifetime, the PET–APPOINTMENT relationship is similarly defined as **one-to-many (1:N)**.

This cascading relational structure ensures that every appointment record in the system can be fully traced back to its associated pet and the owning client through a single query chain. This design directly enables the eager loading strategy employed in the LINQ data access layer, where `.Include()` and `.ThenInclude()` clauses traverse this hierarchy to retrieve fully populated entity graphs in a single optimized database round trip.

```
public class AppDbContext : DbContext
{
    6 references
    public DbSet<Client> Clients { get; set; }
    7 references
    public DbSet<Pet> Pets { get; set; }
    8 references
    public DbSet<Appointment> Appointments { get; set; }
}
```

You, last month • AppDbContext.cs

3.4) DbSet Declarations

The three `DbSet<T>` properties defined within `AppDbContext` serve as the entry points through which Entity Framework Core maps C# model classes to their corresponding database tables, enabling all LINQ query operations described in Section 4 to execute against the live SQLite database.

4) LINQ Queries

Language Integrated Query (LINQ) is heavily utilized in the data access layer of PetClinicApp to write type-safe, readable, and highly optimized database queries. By integrating LINQ with Entity Framework Core, the system translates C# expressions directly into optimized SQL queries, satisfying the project performance benchmarks.

Below are the specific implementations of LINQ operations categorized by their architectural purpose:

4.1) LINQ Filtering & Ordering (Today's Schedule)

This query fetches the specific calendar operations for the current date, satisfying the operational dashboard and staff scheduling requirements.

```
/// Filters today's appointments and orders them by date.
/// (Requirement: LINQ Filtering & Ordering)
0 references
public List<Appointment> GetTodaysAppointments()
{
    using var context = new AppDbContext();
    return context.Appointments
        .Include(a => a.Pet)
        .ThenInclude(p => p.Client) // Include related tables (Join)
        .Where(a => a.AppointmentDate.Date == DateTime.Today) // Filtering
        .OrderBy(a => a.AppointmentDate) // Ordering
        .ToList();
}
```

This method demonstrates complex relational data fetching combined with logical filtering. The `.Include()` and `.ThenInclude()` clauses act as SQL JOIN operations, eagerly loading the related Pet and Client entities to avoid performance bottlenecks. The `.Where()` clause filters the record stream down to the current calendar date (`DateTime.Today`), and `.OrderBy()` ensures the result set is chronologically sorted by time before being evaluated into a list.

4.2) LINQ Search Functionality (Client Registry)

To handle rapid lookups within a clinic database that scales up to thousands of users, a dynamic search pipeline was implemented.

```

/// Searches by client first name or last name.
/// (Requirement: LINQ Search functionality)
0 references
public List<Client> SearchClients(string term)
{
    if (string.IsNullOrEmpty(term)) return GetAllClients();

    using var context = new AppDbContext();
    return context.Clients
        .Where(c => c.FirstName.ToLower().Contains(term.ToLower()) ||
            c.LastName.ToLower().Contains(term.ToLower())) // Filtering
        .OrderBy(c => c.FirstName) // Ordering
        .ToList();
}

```

This query provides flexible search capabilities across the client database. It converts both the stored values and the search token to lowercase using `.ToLower()` to eliminate case-sensitivity issues during validation. The `.Contains()` method compiles into an SQL LIKE query, scanning both first names and last names simultaneously. Finally, the results are sorted alphabetically via `.OrderBy(c => c.FirstName)`.

4.3) LINQ Aggregating (Financial & Operational Reporting)

Data aggregation is required to feed real-time statistical summaries into the administrative desktop interface upon startup.

```

/// Calculates summary data for the clinic (Total earnings and number of pets).
/// (Requirement: LINQ Aggregating)
0 references
public string GetClinicSummary()
{
    using var context = new AppDbContext();

    // LINQ Sum and Count operations
    decimal totalEarnings = context.Appointments.Where(a => a.IsPaid).Sum(a => a.ServiceFee);
    int totalPets = context.Pets.Count();

    return $"There are {totalPets} registered pets in the clinic. Total Earnings: {totalEarnings:C2}";
}

```

This method implements multiple data aggregation tasks using specialized LINQ mathematical extensions. The `.Count()` operator counts the total number of distinct entities present inside the Pets dataset. Concurrently, a filtered aggregation is performed on the Appointments table where `.Where(a => a.IsPaid)` isolates paid transactions, and `.Sum(a => a.ServiceFee)` computes the total cumulative earnings of the facility dynamically.

4.4) LINQ Relational Mapping (One-to-Many Isolation)

This specific query guarantees accurate data fetching, isolating pets that map directly to a selected customer profile.

```
/// Retrieves all pets belonging to a specific client.  
0 references  
public List<Pet> GetPetsByClientId(int clientId)  
{  
    using var context = new AppDbContext();  
    return context.Pets  
        .Where(p => p.ClientId == clientId)  
        .OrderBy(p => p.Name)  
        .ToList();  
}
```

This query acts as the direct operational bridge for the one-to-many relationship defined in the database schema. It utilizes a precise `.Where()` lambda constraint to isolate only the `Pet` entities whose foreign key (`ClientId`) strictly matches the requested primary argument. The return set is systematically organized alphabetically by the pet's name via `.OrderBy()`.

5) Delegates and Events

The architectural communication layer of `PetClinicApp` utilizes asynchronous notification patterns built entirely on C# delegates and events. In cross-platform applications where internal actions within a desktop administrative panel need to broadcast status changes, or web-initiated scheduling requests must trigger immediate visual alerts, standard procedural function calls create rigid coupling. By incorporating an event-driven mechanism, the system core can invoke real-time alerts without needing to know which specific user interface components or platforms are currently listening for the updates.

Below is the implementation of the event-driven notification infrastructure utilized inside the data services layer:

```

// =====
// 4. DELEGATES & EVENTS
// =====

// Delegate definition
2 references
public delegate void ClinicNotifyHandler(string message, object entity);

// Two separate events for new record creation and deletion.
1 reference
public event ClinicNotifyHandler OnAppointmentCreated;
1 reference
public event ClinicNotifyHandler OnAppointmentDeleted;

/// Adds an appointment and triggers an event if successful.
0 references
public void AddAppointmentWithNotification(Appointment appointment)
{
    AddAppointment(appointment);

    // Event triggering (Requirement: Notifications)
    OnAppointmentCreated?.Invoke($" A new appointment has been added for {appointment.AppointmentDate}!", appointment);
}

/// Deletes the appointment and triggers an event if successful.
0 references
public void DeleteAppointmentWithNotification(int appointmentId)
{
    using var context = new AppDbContext();
    var appointment = context.Appointments.Find(appointmentId);

    if (appointment != null)
    {
        DeleteAppointment(appointmentId);

        // Event triggering (Requirement: Triggering updates)
        OnAppointmentDeleted?.Invoke(" Appointment has been successfully removed from the system.", appointment);
    }
}

```

5.1) Architectural Purpose and Functionality

The application establishes a explicit blueprint for messaging using the ClinicNotifyHandler delegate. This signature guarantees that any registered handler method will receive a string message containing human-readable alert content alongside a generic object state tracking entity. This dual-parameter design enables the subscriber to display text-based alerts while simultaneously updating application cache states using the attached entity record payload.

To comply with systemic business monitoring rules, two independent event signals are registered: OnAppointmentCreated and OnAppointmentDeleted. The system encapsulates data modifications within wrapper operations. When a scheduling workflow completes via AddAppointmentWithNotification, the transaction saves down to the database and evaluates the active subscribers on the creation event hook. If an active UI context or event observer is attached, the query triggers the event using the conditional null invocation operator.

The same detached messaging logic governs record removals through DeleteAppointmentWithNotification. Once the query locates the record ID and successfully executes the drop logic, the context alerts the application shell through the deletion event

trigger. This approach maintains platform synchronization between the desktop data grids and the customer web portal views without violating clear layer isolation boundaries.

6) MVC Structure

To fulfill the web-based requirements of the project and ensure clean separation of concerns, the customer portal was developed using the ASP.NET Core MVC (Model-View-Controller) architecture. This structural pattern divides the application into three interconnected components:

- Models and ViewModels: The core business entities (Client, Pet, Appointment) are managed within the Core layer. However, to handle complex UI data transfers safely, dedicated ViewModels such as BookAppointmentViewModel and MyAppointmentsViewModel were implemented. These ViewModels encapsulate validation rules and aggregate data, preventing the direct exposure of raw database models to the client.
- Views: The presentation layer utilizes Razor Pages (.cshtml) paired with Bootstrap 5. The views dynamically render server-side data, such as displaying conditional HTML badges based on an appointment's status (Pending, Completed, Cancelled) using inline C# logic.
- Controllers: The AppointmentController and PetController act as the operational bridge. They intercept HTTP requests (GET/POST), invoke the appropriate LINQ queries from the ClinicService, validate the incoming Model States, and return the populated views. This modularity ensures that the backend processing is completely isolated from the frontend design.

7) Sources

7.1) Source Code Repository (GitHub)

The complete source code for both the ASP.NET Core Web Application and the .NET MAUI Desktop Application is hosted on GitHub.

GitHub Repository Link :

<https://github.com/serhatuzunbayir/PetGroomingSPRING2025-26>

7.2) Installation and Execution Guide (Release Version)

To streamline the evaluation process, the application has been pre-compiled and published as a standalone Release on GitHub. The SQLite database uses a dynamic relative path (Environment.SpecialFolder.MyDocuments), meaning **no manual database configuration is required**. The system will automatically generate the database file upon the first launch.

To run the applications without needing to build the source code, please follow these simple steps:

1. Navigate to the **Releases** section on the provided GitHub repository page.
2. Download the latest PetClinicApp_Release.zip file and extract its contents to your local machine.
3. **Running the Desktop Application:** Open the extracted folder, navigate to the Desktop directory, and double-click PetClinicApp.Desktop.exe to launch the staff administration panel.
4. **Running the Web Application:** Navigate to the Web directory within the extracted folder and double-click PetClinicApp.Web.exe. This will start the local web server. Once the terminal window opens, go to your web browser and navigate to <http://localhost:5000> to access the customer portal.

7.3) Video Demonstrations

The following video links provide a comprehensive, step-by-step demonstration of the system's functionalities, including the real-time database synchronization between the Web and Desktop platforms:

Project Demonstration Video Link:

<https://www.youtube.com/watch?v=HJbbMdClTw>

<https://youtu.be/JFSXcmEq82k>

8) User Interface Design

The user interface (UI) of PetClinicApp was designed with a dual-focus strategy: maximizing data-entry efficiency for clinic staff (Desktop) and providing an intuitive, transparent experience for customers (Web).

Desktop Administration Panel (.NET MAUI): The desktop interface is built around a multi-tabbed AppShell navigation structure. The primary "Dashboard" utilizes real-time CollectionView components bound to LINQ aggregation queries to immediately display "Today's Appointments" and financial summaries upon startup. The UI heavily relies on Grid layouts and data-bound DataTemplates to ensure large datasets (like client registries) are rendered without UI thread blocking.

Web Customer Portal (ASP.NET Core MVC): The web interface was designed using Bootstrap to guarantee full responsiveness across mobile and desktop browsers. A critical UI feature is the "Book Appointment" wizard, which utilizes dynamic AJAX requests. When a user selects their name from a dropdown menu, an asynchronous JavaScript function fetches only that specific user's registered pets from the server, dynamically populating the subsequent dropdown without reloading the page. Furthermore, the "My Appointments" interface uses color-coded rows and status badges (e.g., Green for Completed, Yellow for Pending) to drastically improve visual data parsing for the end-user.

[Home](#)[Book Appointment](#)[My Appointments](#)[Pets](#)[All Appointments](#)



Pet Grooming & Veterinary Clinic

Welcome to the online customer portal. Book appointments, view your pet's records and appointment history.



Book Appointment

Schedule a veterinary or grooming appointment for your pet.

[Book Now](#)



My Appointments

View your past and upcoming appointments, filtered by your name.

[View Mine](#)



Pet Profiles

Browse all registered pets and view their clinical notes and history.

[View Pets](#)



About This Portal

 **Online Appointment Booking** — Veterinary or Grooming

 **Appointment History** — View past & upcoming visits

 For staff management, use the **Desktop Application**

 Real-time notifications when appointments are booked



Book an Appointment

Fill in the form below to schedule a veterinary or grooming appointment for your pet.



Step 1: Select Your Name

Client Name

Test



Step 2: Select Your Pet

Pet

TestDog ()



Step 3: Appointment Details

Appointment Type

Veterinary

Date & Time

06/03/2026 12:00 AM

17

Dashboard

Dashboard Clients Pets Appointments

Pet Clinic Dashboard

Welcome to Pet Grooming & Veterinary Appointment Scheduler



14

Registered Pets



16

Total Clients



\$0,00

Total Earnings



Clinic Summary

There are 14 registered pets in the clinic. Total Earnings: \$0,00

Quick Actions

+ Add New Client

+ Add New Pet

Appointments

Today's Appointments

Appointments

Dashboard Clients Pets Appointments

Appointment Information

Pet *

Select a pet...

TestDog

Date *

2.06.2026

Time *

12

00

00

Type *

Select type...

Veterinary

Service Fee

Enter amount



Paid

Appointment List

Today's Appointments

Show All

All Types

Filter by Type:

TodayPet

Grooming

Date: 02.06.2026 00:00

Fee: \$0,00

Pending

TestDog

Veterinary

Date: 03.06.2026 00:00

Fee: \$500,00

Pending

UpdatePet

Veterinary

Date: 02.06.2026 00:00

Fee: \$0,00

Completed

Today's Appointments

9) Tests

The testing layer of PetClinicApp is implemented using the xUnit framework, targeting the PetClinicApp.Core project through a dedicated PetClinicApp.Tests project. A total of 10 unit tests have been written inside the ClinicServiceTests class, each designed to validate a distinct functional requirement of the system's core business logic in isolation.

9.1) Test Infrastructure

Each test method follows the standard Arrange–Act–Assert pattern to maintain clarity and consistency across the test suite. A fresh ClinicService instance is instantiated at the beginning of every test, ensuring that no shared state or residual database data from a previous test can interfere with the outcome. Where foreign key constraints exist, dependent entities such as Client and Pet are created and persisted prior to the operation under test.

9.2) Test Cases

- Test 1 — AddAppointment (CRUD: Create): Verifies that invoking AddAppointment() with a valid Appointmententity linked to an existing pet increases the total appointment count in the database by exactly one.
- Test 2 — SearchClients (LINQ: Search): Confirms that SearchClients() returns a non-null result set containing at least one client whose FirstName or LastName matches the provided search term, validating the case-insensitive LINQ filtering pipeline.
- Test 3 — GetClinicSummary (LINQ: Aggregation): Asserts that GetClinicSummary() returns a non-empty string that contains the expected keyword "Total Earnings", confirming that the aggregation routine executes and formats its output correctly.
- Test 4 — AddClient (CRUD: Create): Validates that calling AddClient() with a new Client entity increases the total client count retrieved from the database by one, confirming successful persistence.
- Test 5 — DeleteClient (CRUD: Delete): Adds a client record and subsequently calls DeleteClient(), asserting that the count after deletion is exactly one less than the count after insertion, confirming that the delete operation removes precisely the targeted record.
- Test 6 — GetPetsByClientId (LINQ: Filtering): Registers a client along with two associated pets, then calls GetPetsByClientId() and asserts both that the returned list

contains exactly two entries and that every returned pet's ClientId matches the expected owner, validating the one-to-many isolation query.

- **Test 7 — GetTodaysAppointments (LINQ: Filtering):** Inserts an appointment with DateTime.Today as its date and confirms that the record appears in the result of GetTodaysAppointments(), while simultaneously asserting that every entry in the returned list has a date component equal to today's date.
- **Test 8 — GetAllPets with Eager Loading (LINQ: Include):** Creates a client-pet pair and calls GetAllPets(), then retrieves the specific pet by its ID and asserts that its Client navigation property is not null, verifying that the .Include() eager loading clause successfully joins the related entity.
- **Test 9 — UpdateAppointment (CRUD: Update):** Creates an appointment with a Pending status, updates it to Completed via UpdateAppointment(), and then re-fetches the record from the database to assert that the persisted status reflects the change.
- **Test 10 — DeleteAppointment (CRUD: Delete):** Adds an appointment, records the count before deletion, calls DeleteAppointment(), and asserts that the resulting count is exactly one less, confirming that the delete operation functions correctly at the appointment level.

Figure : Test Execution Results - All 10 unit tests successfully passed via .NET CLI.

```
Gerçekleştirildi (0,6sn)
PetClinicApp.Core net10.0 başarılı (0,2sn) → C:\Users\90537\Downloads\PetGroomingSPRING2025-26-main\PetGroomingSPRING2025-26-main\PetClinicApp.Core\bin\Debug\net10.0\PetClinicApp.Core.dll
PetClinicApp.Tests net10.0 başarılı (0,2sn) → bin\Debug\net10.0\PetClinicApp.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4450e68bbb8b (64-bit .NET 10.0.3)
[xUnit.net 00:00:00.11] Discovering: PetClinicApp.Tests
[xUnit.net 00:00:00.17] Discovered: PetClinicApp.Tests
[xUnit.net 00:00:00.20] Starting: PetClinicApp.Tests
[xUnit.net 00:00:02.37] Finished: PetClinicApp.Tests
PetClinicApp.Tests testi net10.0 başarılı (3,4s)

Test Özeti: toplam: 10; başarısız: 0; başarılı: 10; atlandı: 0; süre: 3,4s
*5,2" sn'de başarılı oluşturun
```

10) Conclusion

The PetClinicApp project successfully achieved its core objective: delivering a synchronized, robust, and dual-platform management system for modern veterinary and grooming facilities.

By strictly implementing a decoupled architecture, the system seamlessly bridges the operational gap between clinic administrators and pet owners.

The successful integration of advanced C# mechanisms—such as dynamic LINQ querying for rapid data aggregation, eager loading for complex relational mapping, and Delegate/Event-driven triggers for real-time cross-platform notifications—proves the technical viability of the solution. Backed by a comprehensive suite of xUnit tests verifying database operations, the final product eliminates manual scheduling conflicts, enhances clinical transparency, and provides a highly scalable foundation for future expansions.